

# Unit 8

# Loop Unrolling and Nested Loops

---

ECE-GY 6463: ADVANCED HARDWARE DESIGN

PROFS. SUNDEEP RANGAN, SIDDHARTH GARG

# Learning Objectives

---

- ❑ How to **unroll** loops in Vitis HLS
- ❑ Mitigate memory bottlenecks in unrolled loops with:
  - Increased memory bit width
  - **Memory partitioning**
- ❑ Design efficient **nested loops**

# Outline

---

## Loop Unrolling and Memory Constraints

- ☐ Nested Loops

- ☐ FIR Filter Example

# Loop Unrolling

- ❑ Simple idea: Repeat processing in each cycle
- ❑ Unroll by 2
  - Need half the number of iterations
  - But double the logic

```
// Normal loop  
for (i=0; i < N; i++) {  
    y[i] = w[i]*x[i];  
}
```

```
// Unrolled by factor of 2  
for (i=0; i < N; i+=2) {  
    y[i] = w[i]*x[i];  
    y[i+1] = w[i+1]*x[i+1];  
}
```

# Loop Unrolling in Vitis

---

- ❑ Can be done with a simple pragma
- ❑ No need to manually unroll!
- ❑ Handles cases when  $N$  is not divisible by unroll factor

```
// Vitis pragma
for (i=0; i < N; i++) {
    #pragma HLS unroll factor=2
    y[i] = w[i]*x[i];
}
```

=

```
// Unrolled by factor of 2
for (i=0; i < N; i+=2) {
    y[i] = w[i]*x[i];
    y[i+1] = w[i+1]*x[i+1];
}
```

# Memory Bottleneck

*// Stage 0: Two reads from x and w*

```
x1_s0 <= x[i];  
x2_s0 <= x[i+1];  
w1_s0 <= w[i];  
w2_s0 <= w[i+1];
```

*// Stage 1: Two products -> OK*

```
y1_s1 <= x1_s0 * w1_s0;  
y2_s1 <= x2_s0 * w2_s0;
```

*// Stage 2: Two writes*

```
y[i] <= y1_s1;  
y[i+1] <= y2_s1;  
i <= i + 2;
```

## ❑ Memory bottleneck

- Loop unrolling is often limited by memory accesses

## ❑ Consider vector multiply SV code

## ❑ Stage 0 needs:

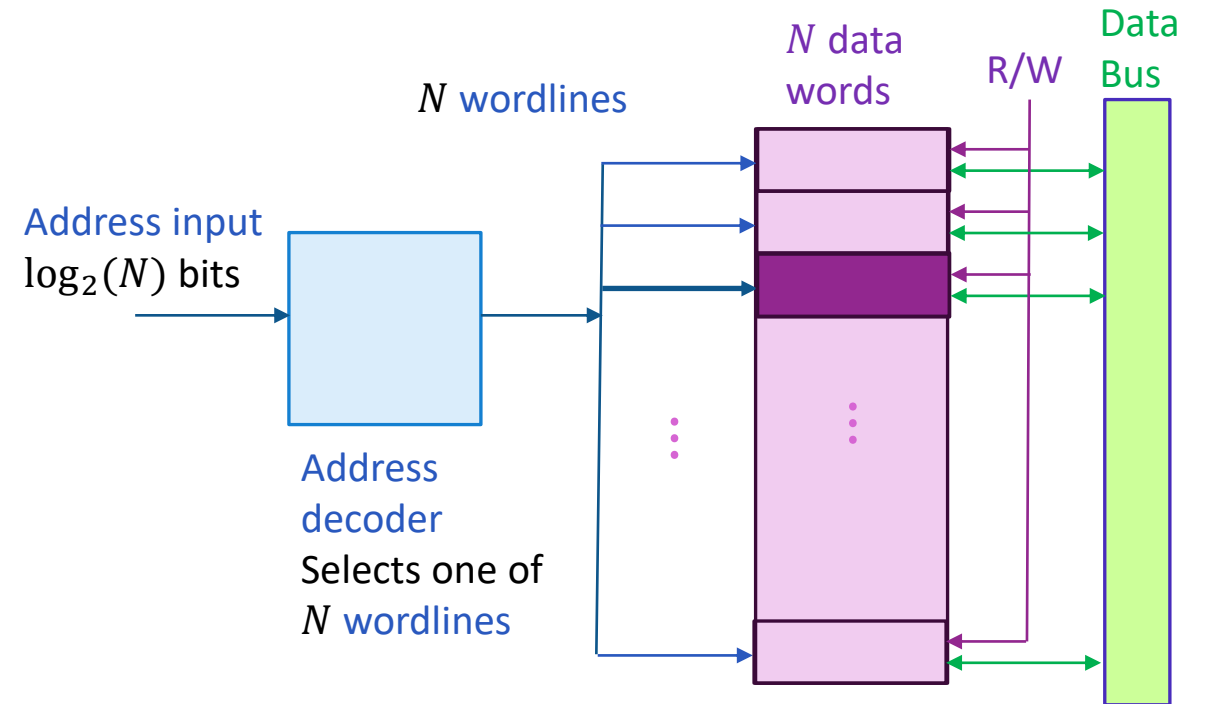
- Two reads from  $x$  and two reads from  $w$

## ❑ Stage 2 needs:

- Two writes to  $y$

# Single Port Memory

- Recall memory architecture from Unit 4
- Basic memories are **single port**
- There is a single address decoder
- Can read / write one word in a cycle



# Three Solutions

---

## ❑ Solution 1: Use a true multi-port RAM

- Most general. Allows read and write from any combination of addresses
- Xilinx FGPA BRAMs are two port
- But most costly in hardware

## ❑ Solution 2: Use a wider bitwidth

- Say 64 bits to read two 32 bit numbers
- Works only when the accesses are contiguous

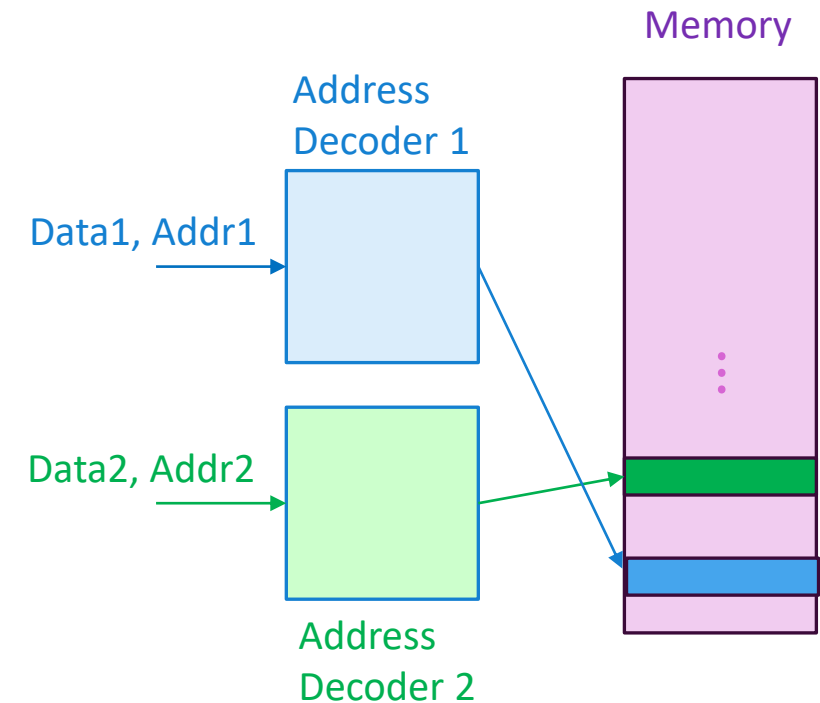
## ❑ Solution 3: Bank or partition memory

- Divide memory into partitions
- Each partition has its own decoder
- Works if accesses can be across different partitions

# Mutli-Port RAMs

- ❑ Allow arbitrary read / writes
  - Except for read-write collisions to same address in same cycle
- ❑ BRAM in FPGAs generally only support true two port
- ❑ BMutli-port SRAMs in ASICs are expensive

| Logical ports        | Typical cell style           | Rough transistors/bit | Rough area vs 6T       |
|----------------------|------------------------------|-----------------------|------------------------|
| 1R/1W (single-port)  | 6T SRAM cell                 | ~6T                   | 1.0×                   |
| 1R/1W (2-port: 1R1W) | 8T cell (separate read path) | ~8T                   | ≈1.3×                  |
| 2R/1W                | 10T–12T cell                 | ~10–12T               | ≈1.7–2.0×              |
| 2R/2W                | 12T–16T+ cell                | ~12–16T+              | ≈2–2.7×                |
| N-port (general)     | custom multi-port SRAM       | ≈6T + O(N·ports)      | grows ~linear in ports |



# Using Wider Bit widths

## Narrow bit width design

- Access only one word per cycle
- Memory bottleneck

```
// Unrolled by factor of 2
for (i=0; i < N; i+=2) {
    y[i] = w[i]*x[i];
    y[i+1] = w[i+1]*x[i+1];
}
```

```
ap_uint<64> xwords[N/2], wwords[N/2], ywords[N/2];

for (i=0; i < N; i += 2) {

    // Read two words from x and w
    xword = xwords[i/2];
    wword = wwords[i/2];

    // Extract values
    x1 = xword.range(31,0);
    x2 = xword.range(63,32);
    w1 = wword.range(31,0);
    w2 = wword.range(63,32);

    // Compute products
    y1 = w1*x1;
    y2 = w2*x2;

    // Write outputs
    yword.range(31,0) = y1;
    yword.range(63,32) = y2;
    ywords[i/2] = yword;
}
```

## Wide bit width design

- Only one R/W to each array
- Can pipeline with II=1
- But need accesses to be contiguous

# Mutli-Port RAMs

- ❑ Allow arbitrary read / writes
  - Except for read-write collisions to same address in same cycle
- ❑ BRAM in FPGAs generally only support true two port
- ❑ BMutli-port SRAMs in ASICs are expensive

